

INTERVIEW PREPARATION GUIDE

Revolut Java

Round 2

Live coding (30–35 min) · Technical conversation (15–20 min)

Concurrency · Databases · Architecture · Scalability · Reliability · Delivery

Java 21

PostgreSQL / jOOQ

Redis

Kubernetes / GCP

DDD · CQRS · TDD

Clean Architecture

Event-Driven

Microservices

Contents

1	Concurrency & Thread Safety	synchronized · Locks · Atomics · Virtual Threads
2	Databases & Data Consistency	ACID · Isolation levels · Indexing · Distributed data
3	Architecture Patterns	DDD · CQRS · Microservices · Event-Driven
4	Scalability & Distributed Systems	CAP · Caching · Sharding · Rate limiting
5	Reliability & Failure Handling	Circuit Breaker · Idempotency · Observability
6	Delivery & Production	Deployments · DB migrations · Kubernetes · Graceful shutdown
7	Live Coding Tips	TDD approach · Java 21 idioms · AI tool use

Concurrency & Thread Safety

Revolut's highest-traffic paths (payments, FX) demand correct concurrent access. Expect both theory and live coding on this topic.

Core primitives

synchronized · **Lock** · **volatile**
· **Atomic***

Higher-level APIs

ExecutorService ·
CompletableFuture · **ForkJoin**

Java 21

Virtual Threads (Project Loom)

Synchronization primitives — trade-off table

Mechanism	Use when	Trade-off
<code>synchronized</code>	Simple mutual exclusion; low contention	Coarse; can't interrupt waiting threads
<code>ReentrantLock</code>	Need <code>tryLock</code> , timed lock, or fairness	Must release in finally block — easy to forget
<code>ReadWriteLock</code>	Many reads, rare writes (e.g. FX rates cache)	Write starvation if reads dominate
<code>StampedLock</code>	Optimistic reads (check then read, retry on write)	Not reentrant; complex API
<code>AtomicLong/Reference</code>	Lock-free counter or reference updates	Only single-variable atomicity
<code>volatile</code>	Visibility guarantee, no atomicity needed	Not for compound actions (check-then-act fails)
<code>ConcurrentHashMap</code>	Concurrent map with segment-level locking	Compound actions (<code>putIfAbsent</code>) still need care

Race condition — classic payment example

```
// WRONG — race condition on balance check
if (account.getBalance() >= amount) {    // Thread A reads 100
    account.debit(amount);                // Thread B also reads 100 — both debit!
}

// CORRECT — atomic compare-and-debit with AtomicLong CAS loop
private final AtomicLong balance = new AtomicLong(10000L);

public boolean debitSafe(long amount) {
    long current;
    do {
        current = balance.get();
        if (current < amount) return false;
    } while (!balance.compareAndSet(current, current - amount));
    return true;
}
```

```
// SIMPLE – synchronized method (start here in interviews)
public synchronized boolean debit(long amount) {
    if (balance < amount) return false;
    balance -= amount;
    return true;
}
```

Deadlock — causes and resolution

```
// DEADLOCK: Thread 1 locks A then B; Thread 2 locks B then A

// FIX 1: consistent lock ordering (always lock lower id first)
void transfer(Account from, Account to, long amount) {
    Account first = from.id() < to.id() ? from : to;
    Account second = from.id() < to.id() ? to : from;
    synchronized (first) {
        synchronized (second) {
            from.debit(amount);
            to.credit(amount);
        }
    }
}

// FIX 2: tryLock with timeout (ReentrantLock) – back off and retry
if (lock1.tryLock(50, MILLISECONDS) && lock2.tryLock(50, MILLISECONDS)) {
    try { /* transfer */ } finally { lock2.unlock(); lock1.unlock(); }
} else { /* retry with backoff */ }
```

Virtual Threads (Java 21) — key for Revolut's stack

```
// Platform thread: expensive (~1MB stack, OS-scheduled)
// Virtual thread: cheap (JVM-scheduled, millions concurrently)
try (ExecutorService ex = Executors.newVirtualThreadPerTaskExecutor()) {
    for (var payment : payments) {
        ex.submit(() -> processPayment(payment));
    }
}

// CAUTION: synchronized blocks pin the carrier thread – use ReentrantLock instead
```

Interview tip: Always start with `synchronized` (simplest), then offer `ReentrantLock` for fairness/timeout, then `CAS/AtomicLong` for lock-free throughput. Say: "I'd profile first — premature lock optimisation is a common trap."

Databases & Data Consistency

Revolut runs PostgreSQL via jOOQ. Expect deep questions on transactions, isolation, indexing, and distributed data patterns.

ACID properties

ATOMICITY

All operations succeed or all fail. A payment debit + credit must never be half-committed.

CONSISTENCY

Transaction brings DB from one valid state to another. Balance ≥ 0 constraints never violated mid-transaction.

ISOLATION

Concurrent transactions behave as if sequential. Dirty reads and phantom reads controlled by isolation level.

DURABILITY

Committed transactions survive crashes. PostgreSQL's Write-Ahead Log (WAL) guarantees this.

Isolation levels

Level	Dirty read	Non-repeatable read	Phantom read	Use in fintech
READ UNCOMMITTED	Possible	Possible	Possible	Never — too dangerous
READ COMMITTED	Prevented	Possible	Possible	Default; most OLTP queries
REPEATABLE READ	Prevented	Prevented	Possible	Balance snapshot reads
SERIALIZABLE	Prevented	Prevented	Prevented	Fraud detection; critical transfers

Revolut context: PostgreSQL default is READ COMMITTED. For double-spend prevention use `SELECT FOR UPDATE` (pessimistic) or an optimistic `version` column check.

```
-- Pessimistic locking: SELECT FOR UPDATE
BEGIN;
SELECT balance FROM accounts WHERE id = 42 FOR UPDATE;
UPDATE accounts SET balance = balance - 100 WHERE id = 42 AND balance >= 100;
COMMIT;

-- Optimistic locking: version column (no DB lock, retry on conflict)
UPDATE accounts
SET balance = balance - 100, version = version + 1
WHERE id = 42 AND version = :expectedVersion AND balance >= 100;
-- rowsAffected == 0 means concurrent modification → retry
```

Indexing essentials

- **B-tree index:** default; equality + range queries. Use on `user_id`, `created_at`.
- **Partial index:** `WHERE status = 'PENDING'` — only indexes hot rows; huge win for processing queues.
- **Composite index:** column order matters — most selective or most frequently queried column first.
- **EXPLAIN ANALYZE:** always verify the planner is using your index; watch for sequential scans on large tables.
- **Index bloat:** high-write tables accumulate dead tuples — schedule regular `VACUUM` in PostgreSQL.

Distributed data patterns

Challenge	Solution	Trade-off
Cross-service transaction	Saga (choreography or orchestration)	Eventual consistency; requires compensating transactions
Duplicate payment on retry	Idempotency key + unique DB constraint	Extra storage; need key expiry policy
Distributed counter (daily limits)	Redis INCR + TTL, or row with <code>SELECT FOR UPDATE</code>	Redis: fast but approximate; DB: consistent but slower
Read scalability	Read replicas + CQRS (separate read model)	Replication lag; reads may be stale

Key phrase: "For money movement I'd use `SELECT FOR UPDATE` or optimistic versioning to prevent double-spend. Cross-service transfers are a classic 2PC problem — at Revolut's scale, a Saga with compensating transactions is more resilient than distributed 2PC."

Architecture Patterns

Revolut explicitly uses DDD, CQRS, Clean Architecture, microservices, and event-driven patterns. Expect design questions framed around payments and accounts.

Domain-Driven Design (DDD)

AGGREGATE

Cluster of objects treated as a single unit.

`Account` aggregate owns `Balance` and `Transaction` history. Always load and save the whole aggregate.

AGGREGATE ROOT

Single entry point into an aggregate. External code only holds a reference to the root, never to inner entities directly.

BOUNDED CONTEXT

Explicit boundary where a domain model applies. "Account" means something different in Payments vs KYC — keep them separate with explicit translation layers.

DOMAIN EVENT

`MoneyDebited`, `PaymentProcessed` — immutable facts that happened. Drive decoupled reactions across bounded contexts.

```
// Aggregate root – business rules live inside the domain object
public class Account {
    private final AccountId id;
    private Money balance;
    private final List<DomainEvent> events = new ArrayList<>();

    public void debit(Money amount, TransactionId txId) {
        if (balance.isLessThan(amount))
            throw new InsufficientFundsException(id);
        balance = balance.subtract(amount);
        events.add(new MoneyDebited(id, txId, amount, Instant.now()));
    }

    public List<DomainEvent> pullEvents() {
        var copy = List.copyOf(events);
        events.clear();
        return copy;
    }
}
```

CQRS — Command Query Responsibility Segregation

Separates write (command) and read (query) models. Commands enforce business rules; queries serve read-optimised projections.

COMMAND SIDE (WRITE)

- Commands: `DebitAccountCommand`, `CreatePaymentCommand`

QUERY SIDE (READ)

- Queries: `GetTransactionHistory`, `GetBalance`

- Validates all business rules
- Persists to normalised, ACID write DB
- Emits domain events on success

- Reads from denormalised projection
- Can use separate DB, Redis, or Elasticsearch
- Eventually consistent with write side

Event-driven architecture

```
// Choreography: services react to events autonomously (Kafka)
PaymentsService → emits PaymentInitiated
FraudService     → consumes PaymentInitiated, emits FraudChecked
LedgerService    → consumes FraudChecked (approved), posts to ledger
NotifyService    → consumes PaymentCompleted, sends push notification

// Orchestration: central saga orchestrator drives the flow
PaymentSaga.start()
  → call FraudService.check()      (REJECT → compensate, end)
  → call LedgerService.debit()     (FAIL   → compensate, end)
  → call LedgerService.credit()
  → emit PaymentCompleted
```

Pattern	Pros	Cons
Choreography	Decoupled; each service autonomous	Hard to trace end-to-end; implicit event coupling
Orchestration	Explicit flow; easy to monitor and reason about	Orchestrator becomes bottleneck; central coupling

Key insight: "CQRS and event sourcing pair naturally — the write side stores events as the source of truth; the read side rebuilds projections. For a transaction ledger this gives full auditability with zero data loss — critical in regulated fintech."

Scalability & Distributed Systems

Revolut serves 70M+ customers. Questions probe how systems behave under load and across distributed environments.

Vertical scaling

Bigger machine — simple but has ceiling + SPOF

Horizontal scaling

More instances — handles spikes; requires stateless design

Stateless services

Session/state in Redis → any instance can serve any request

CAP theorem

Pick two	What you sacrifice	Example
CP (Consistency + Partition tolerance)	Availability — refuse requests when uncertain	ZooKeeper, HBase, PostgreSQL
AP (Availability + Partition tolerance)	Consistency — may return stale data	Cassandra, DynamoDB (eventual)
CA (Consistency + Availability)	Partition tolerance — impossible in distributed systems	Single-node RDBMS only

For payments, Revolut needs strong consistency (no double-spend). PostgreSQL with `SERIALIZABLE` isolation provides CP. Redis is used as a cache, not as the source of truth.

Rate limiting — essential in fintech APIs

```
// Token bucket via Redis INCR + TTL — allows controlled bursts
long current = redis.incr("rate:user:" + userId + ":" + currentMinute());
if (current == 1) redis.expire(key, 60);
if (current > LIMIT) throw new RateLimitExceededException();

// Trade-offs:
// Fixed window — fast, but burst at boundary (minute:59 + minute:00)
// Sliding log — exact, but high memory (store each request timestamp)
// Token bucket — best balance: smooth bursts, low memory
```

Caching strategies

Strategy	How it works	Risk
Cache-aside	App loads cache miss from DB, writes to cache	Cache stampede on cold start
Write-through	Write to cache and DB atomically	Higher write latency
Write-behind	Write cache first, async flush to DB	Data loss on crash before flush
Read-through	Cache handles DB fetch transparently	Cold start latency spikes

```
// Cache stampede prevention: distributed lock on cache miss
String rate = redis.get("fx:GBPEUR");
if (rate == null) {
    if (redis.setnx("fx:GBPEUR:lock", "1", 5, SECONDS)) { // only one thread fetches
        rate = fxProvider.getRate("GBPEUR");
        redis.setex("fx:GBPEUR", 30, rate);
        redis.del("fx:GBPEUR:lock");
    } else {
        rate = pollWithBackoff("fx:GBPEUR"); // others wait for populated cache
    }
}
```

Database sharding & partitioning

- **Horizontal sharding:** partition rows by key (e.g. `user_id % N`). Cross-shard queries are expensive — design to avoid them.
- **Consistent hashing:** adding/removing nodes only rebalances $1/N$ of keys — used in distributed caches like Redis Cluster.
- **PostgreSQL partitioning:** range partition on `created_at` for time-series data (transactions); old partitions archived or dropped cheaply.

Reliability & Failure Handling

In regulated fintech a payment failure has real money consequences. Expect questions on defence-in-depth and failure mode design.

Failure patterns and mitigations

Failure mode	Pattern	Implementation
Dependent service down	Circuit Breaker	Resilience4j: CLOSED → OPEN after N failures → HALF_OPEN to probe
Cascading overload	Bulkhead	Separate thread pools per dependency; failure in one doesn't starve others
Slow dependency	Timeout + fallback	<code>CompletableFuture.orTimeout()</code> ; return cached or degraded response
Transient errors	Retry with exponential backoff + jitter	Resilience4j Retry; jitter prevents thundering herd on recovery
Duplicate processing on retry	Idempotency	Idempotency key in request; unique DB constraint; dedup table in Redis

```
// Circuit breaker with Resilience4j
CircuitBreakerConfig config = CircuitBreakerConfig.custom()
    .failureRateThreshold(50)
    .waitDurationInOpenState(Duration.ofSeconds(30))
    .slidingWindowSize(10)
    .build();
CircuitBreaker cb = CircuitBreakerRegistry.of(config).circuitBreaker("fraud-check");

Supplier<FraudResult> decorated = CircuitBreaker.decorateSupplier(cb, fraudService::check);
Try.ofSupplier(decorated)
    .recover(ex -> FraudResult.ALLOW_WITH_REVIEW); // graceful fallback

// Idempotent payment handler
public PaymentResult processPayment(PaymentRequest req) {
    var existing = idempotencyStore.get(req.idempotencyKey());
    if (existing.isPresent()) return existing.get(); // replay same result

    var result = doProcess(req);
    idempotencyStore.save(req.idempotencyKey(), result, Duration.ofDays(7));
    return result;
}
```

Observability

METRICS

P99 latency, error rate, queue depth, cache hit rate. Alert on SLO breaches (e.g. P99 payment latency > 500ms).

DISTRIBUTED TRACING

Trace ID propagated through all services (OpenTelemetry). Essential to diagnose a failed payment across 5 microservices.

STRUCTURED LOGGING

JSON logs with `traceId`, `userId`, `paymentId` fields. Enables fast querying in GCP Logging / Splunk.

HEALTH CHECKS

Readiness: DB + cache healthy. Liveness: JVM responsive. K8s uses these to route traffic and restart broken pods.

Graceful degradation — design in levels

- **Full service:** all features available, all dependencies healthy.
- **Degraded:** FX provider slow → use cached rates with a stale-data warning to the user.
- **Minimal:** fraud service down → allow low-risk transactions, queue high-risk for manual review.
- **Read-only:** DB write outage → serve balance/history from read replica; queue mutations for replay.

Key phrase: "I design for failure as a first-class concern — every external call gets a timeout, retry budget, and fallback. I'd use Resilience4j's circuit breaker to prevent a slow fraud-check from blocking the entire payment flow."

Delivery & Production

Revolut runs on Kubernetes + GCP. Expect questions on zero-downtime deployments, safe DB migrations, and operational readiness.

Deployment strategies

Strategy	How	Risk	Best for
Rolling update	Replace pods one by one; Kubernetes default	Brief mix of v1 and v2 handling requests	Backward-compatible changes
Blue/Green	Two identical envs; flip load balancer	Double infrastructure cost; instant rollback	Major releases; DB migrations
Canary	Route 5% traffic to new version; ramp up	Complex routing; need metrics to decide rollout	High-traffic, risk-sensitive paths
Feature flag	Deploy code off, enable per user/cohort	Flag debt accumulates if not cleaned up	Decoupling deploy from release

Database migrations — zero downtime (Flyway)

```
// Rule: migrations must be backward-compatible across the deploy window
// Adding a column safely – 3-phase deploy:

-- Phase 1 (migration): add NULLABLE column with DEFAULT
ALTER TABLE payments ADD COLUMN fee_amount BIGINT DEFAULT 0;

-- Phase 2 (code): read new column if present, write both old and new fields

-- Phase 3 (cleanup): enforce NOT NULL once all rows are populated
ALTER TABLE payments ALTER COLUMN fee_amount SET NOT NULL;

-- NEVER: rename or drop columns while old code is still running in production
```

Kubernetes essentials

READINESS PROBE

K8s only routes traffic to pods that return 200. Declare ready only when DB and cache connections are healthy.

LIVENESS PROBE

K8s restarts pods that fail this check. Detects deadlocks or OOM states that don't crash the JVM outright.

HPA (HORIZONTAL POD AUTOSCALER)

Scales pods on CPU or custom metrics (e.g. Kafka queue depth). Requires correct resource requests and limits set.

PODDISRUPTIONBUDGET

Guarantees a minimum number of healthy pods during voluntary disruptions (node drain, rolling updates).

Graceful shutdown in Java

```
// Spring Boot: server.shutdown=graceful
// spring.lifecycle.timeout-per-shutdown-phase=30s
// K8s sends SIGTERM → Spring drains in-flight requests → process terminates

Runtime.getRuntime().addShutdownHook(new Thread(() -> {
    log.info("Draining {} in-flight requests", inFlight.get());
    executor.shutdown();
    try {
        executor.awaitTermination(30, TimeUnit.SECONDS);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}));
```

For every deployment question mention: "I'd use canary deployments with automatic rollback triggers on error rate spikes — never push directly to 100% traffic on a payment-critical service."

Live Coding Tips

30–35 minutes, multiple stages. Speed, correctness, and communication matter equally. Minimum 2–3 stages must be completed.

Framework for every coding interview

1. **Verbalize the problem** — restate requirements in your own words. Confirm edge cases: "What should happen if the balance equals exactly the debit amount?"
2. **Plan before typing** — say "I'll implement a simple synchronized solution first, then we can discuss lock-free alternatives."
3. **Write a failing test first (TDD)** — shows intent, documents expected behaviour, catches bugs before they hide.
4. **Implement minimum viable solution** — make the test pass simply. Avoid over-engineering stage 1.
5. **Add edge-case tests** — null inputs, boundary values, concurrent access.
6. **Discuss trade-offs proactively** — "This is $O(n)$ — if the list is large I'd sort + binary search for $O(\log n)$."

TDD pattern — payment service example

```
// Stage 1: basic business logic
@Test
void debit_reducesBalance() {
    var account = new Account("acc-1", Money.of(100, GBP));
    account.debit(Money.of(30, GBP));
    assertThat(account.getBalance()).isEqualTo(Money.of(70, GBP));
}

@Test
void debit_throwsWhenInsufficientFunds() {
    var account = new Account("acc-1", Money.of(20, GBP));
    assertThatThrownBy(() -> account.debit(Money.of(30, GBP)))
        .isInstanceOf(InsufficientFundsException.class);
}

// Stage 2: thread safety
@Test
void concurrentDebits_neverResultInNegativeBalance() throws Exception {
    var account = new Account("acc-1", Money.of(1000, GBP));
    var executor = Executors.newFixedThreadPool(20);
    var latch = new CountDownLatch(100);

    for (int i = 0; i < 100; i++) {
        executor.submit(() -> {
            try { account.debit(Money.of(20, GBP)); }
            catch (InsufficientFundsException e) {}
            finally { latch.countDown(); }
        });
    }
    latch.await();
}
```

```

    });
}
latch.await(5, SECONDS);
assertThat(account.getBalance().amount()).isGreaterThanOrEqualTo(0);
}

```

Typical stage breakdown

STAGE 1

Core business logic — single-threaded operations, happy path and error cases.
Clean code, value objects.

Target: 8 min

STAGE 2

Add concurrency — make the same code thread-safe. Pick your mechanism and justify it with trade-offs.

Target: 10 min

STAGE 3

Add persistence or APIs — introduce a repository, handle idempotency, or improve performance.

Bonus points

Java 21 idioms worth using

```

// Records for value objects – immutable, no boilerplate
record Money(long amount, Currency currency) {
    Money { if (amount < 0) throw new IllegalArgumentException("Negative amount"); }
    Money add(Money other) { return new Money(amount + other.amount, currency); }
    Money subtract(Money other) { return new Money(amount - other.amount, currency); }
    boolean isLessThan(Money other) { return amount < other.amount; }
}

// Sealed interfaces for explicit result types (avoids null/exception as control flow)
sealed interface DebitResult
    permits DebitResult.Success, DebitResult.InsufficientFunds {}
record Success(Money newBalance) implements DebitResult {}
record InsufficientFunds(Money avail) implements DebitResult {}

// Pattern matching switch – exhaustive, clean
return switch (account.debit(amount)) {
    case Success(Money b)          -> ResponseEntity.ok(b);
    case InsufficientFunds(Money av) -> ResponseEntity.unprocessableEntity().body(av);
};

```

On AI tool use during the interview

Use AI autocomplete for boilerplate (imports, test scaffolding, getters). Narrate what you are generating: *"I'll use autocomplete for the JUnit 5 setup."* Be ready to explain, modify, and debug any AI-generated code instantly. Never paste a block without reading it — the interviewer will ask about every line.

Quick reference — Revolut's tech stack

